



**FACULTY OF
NUCLEAR SCIENCES
AND PHYSICAL
ENGINEERING
CTU IN PRAGUE**

Hands-on Machine Learning - XGBoost

SZD2

Karla Žertová



Machine Learning



Signal Detection as ML Task

Input data:

- Tagged simulated data

Model:

- Supervised learning
- BDTs with XGBoost

Output:

- Invariant mass plot

Supervised Learning

Supervised learning is a machine learning approach in which a model learns a mapping

$$X \rightarrow y$$

from examples with known labels.

In classification:

- X : input features of a candidate
- y : target label
 - $y = 1$: signal
 - $y = 0$: background

The model is trained on labeled examples and then applied to unseen data to predict how signal-like each candidate is.

Supervised Learning in Particle Reconstruction

In particle physics, each reconstructed candidate is described by a set of variables (p_T , DCA, CPA, ...)

Goal: use these variables to distinguish

- true particle decays (signal)
- random combinations of tracks (background)

Why use supervised learning?

- many variables are correlated
- simple rectangular cuts may be inefficient
- ML can learn more powerful decision boundaries

Loss Function

During training, the model parameters are adjusted to minimize a **loss function**.

General idea:

$$\mathcal{L} = \mathcal{L}(y, \hat{y})$$

where y is the true label and $\hat{y}$ is the model prediction

The loss quantifies how wrong the prediction is:

- small loss = good prediction
- large loss = bad prediction

Training consists of finding model parameters that minimize the total loss over all training examples.

Log-Loss for Binary Classification

For binary classification, a common choice is the **log-loss** (binary cross-entropy):

$$\mathcal{L}_{\log} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where:

- $y_i \in \{0, 1\}$ is the true label
- p_i is the predicted probability that the event is signal

Interpretation:

- correct and confident prediction $\rightarrow$ small loss
- wrong and confident prediction $\rightarrow$ very large loss

Why Log-Loss is Useful in Particle Physics

In particle reconstruction, we usually want a model output that can be interpreted as a **signal probability score**.

Log-loss is useful because:

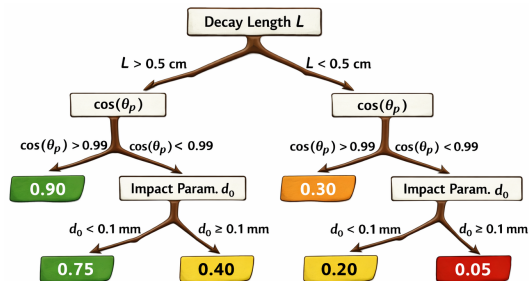
- it rewards correct probability estimates
- it strongly penalizes confidently wrong predictions
- it provides a smooth objective for optimization

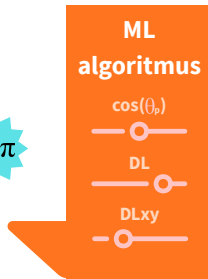
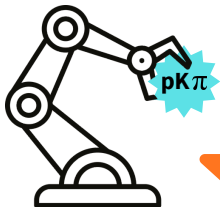
Boosted Decision Trees (BDTs)

- Supervised learning: model trained on labeled data
- Decision tree $T(x)$ splits data based on input variables
- Boosting: combination of many trees to improve performance (α_m same for all trees)

$$\text{BDT}(x) = \sum_{m=1}^M \alpha_m T_m(x)$$

- BDTs used to classify candidates as signal or background
- Example: XGBoost (python)





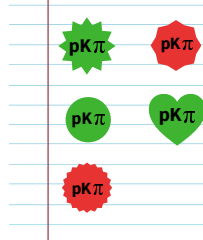
Signal



Background



Labels



Hands-on



Training and Testing Data

Goal: evaluate how well the model generalizes to unseen data

- **Training set:**
 - used to train the model
 - model learns patterns from these data
- **Test set:**
 - not seen during training
 - used to evaluate performance

Training and Testing Data

Why is this important?

- prevents **overfitting** (memorizing instead of learning)
- ensures realistic performance estimate

In particle physics:

- we want a model that works on real data, not only on training samples
- test set mimics independent data

Task 1

To Do: Define how much of all data should go to testing data (the rest goes into training data).

Task 1

Answer: 0.2-0.3



Key XGBoost Parameters

1. Number of trees: `n_estimators`

- Number of boosting steps (trees)
- More trees → more expressive model
- Too many → overfitting

2. Tree depth: `max_depth`

- Controls complexity of each tree
- Small depth (2–4) → simple, robust
- Large depth → captures details but risks overfitting

Key XGBoost Parameters

3. Learning rate: `learning_rate`

- How much each tree contributes
- Small value $\rightarrow$ slow, stable learning
- Large value $\rightarrow$ fast but unstable

How do we choose them?

- Tune using validation/test performance
- Typical strategy:
 - smaller learning rate + more trees
 - shallow trees (depth $\sim 3-5$)

Task 2

To Do: Define how many trees with how many branches will be used and what will be the contribution of each tree.

Task 2

Answer:

- `n_estimators = 100`
- `max_depth = 3`
- `learning_rate = 0.1`

Model Evaluation: Accuracy

Accuracy

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of events}}$$

- Measures overall fraction of correctly classified events
- Simple and intuitive
- **Limitation:** can be misleading for imbalanced data
 - e.g. mostly background → high accuracy without finding signal

Model Evaluation: AUC

AUC (Area Under ROC Curve)

- Measures how well the model separates signal from background
- Based on the ROC curve (signal efficiency vs. background efficiency)
- Range:
 - 0.5 $\rightarrow$ random guessing
 - 1.0 $\rightarrow$ perfect separation

ROC Curve

ROC (Receiver Operating Characteristic) curve

- Shows performance of a classifier for all possible score cuts
- Plots:
 - x-axis: False Positive Rate (FPR) = background efficiency
 - y-axis: True Positive Rate (TPR) = signal efficiency

Interpretation

- Each point = one choice of ML score cut
- Moving along the curve:
 - higher TPR $\rightarrow$ keep more signal
 - higher FPR $\rightarrow$ also keep more background

ROC Curve

Diagonal (straight) line

- Represents a **random classifier**
- Signal and background are indistinguishable
- $AUC = 0.5$

Good model

- Curve bends toward top-left corner
- High signal efficiency with low background contamination

Feature Importance

Feature importance

- Quantifies how much each input variable contributes to the model
- In XGBoost:
 - based on how often and how effectively a feature is used in tree splits

Interpretation

- higher importance → stronger contribution to signal/background separation
- lower importance → less useful feature

Caution

- importance does not imply causality
- correlated variables may share importance

ML Score Distribution

What is shown?

- Distribution of the ML score for:
 - signal vs. background
 - training vs. test samples

ML score

- Output of the model: $p(\text{signal}|X) \in [0, 1]$
- High score $\rightarrow$ signal-like
- Low score $\rightarrow$ background-like

ML Score Distribution

Interpretation of the plot

- Good model:
 - signal peaks near 1
 - background peaks near 0
- Separation between the two distributions indicates performance

Train vs. test

- Compare shapes to check for overfitting
- Large differences $\rightarrow$ model may not generalize well

Working Point

Working point (cut)

- Vertical line = chosen threshold (e.g. $\text{score} > 0.8$)
- Keeps mostly signal while rejecting background

Task 3

Overfit the model: The training data it will predict well but the test data will be predicted poorly.



**FACULTY OF
NUCLEAR SCIENCES
AND PHYSICAL
ENGINEERING
CTU IN PRAGUE**